



StringVariable

🕒 Created time	June 20, 2026 1:06 PM
☰ Category	CommonVariables
👤 Last edited by	 R.A R.A
🕒 Last updated time	August 2, 2026 4:54 AM

```
string firstName = string.Empty;
string lastName = string.Empty;
string filePath = string.Empty;

firstName = "Tim";
lastName = "Corey";
//filePath = "C:\\Temp\\Demo";
filePath = @"C:\Temp\Demo";
//firstName = "123";

//Console.WriteLine(firstName + " " + lastName);

string testString = $"The file for{firstName} is at C:\Sam
pleFile";

//string interpolation
Console.WriteLine($"Hello{firstName}{lastName}");
Console.WriteLine(filePath);
Console.WriteLine(testString);
```

דוגמאות לעבודה עם Strings

הסברים

Variable-ה

ה-Variable הוא מקום בזיכרון שמחזיק מידע כלשהו, כל עוד המידע מתאים ל-Type שהגדרנו עבורו.

מוסכמות שמות Camel Case

ב-Camel Case:

- האות הראשונה של המילה הראשונה היא קטנה.
- האות הראשונה של כל מילה שבאה אחריה היא גדולה.

לדוגמה:

```
stringfirstName = string.Empty;
```

```
stringmyFirstName = string.Empty;
```

או:

אם יש רק מילה אחת, כותבים אותה באותיות קטנות בלבד:

```
stringtest = string.Empty;
```

למה חשוב לתת שמות נכונים למשתנים

יש דוגמה מפורסמת לאתר בשם **Experts Exchange**.

הכוונה הייתה לשם:

```
Experts Exchange
```

אבל בכתובת האתר הוא נכתב כך:

```
expertsexchange
```

ואנשים קראו אותו בטעות כך:

```
expert sex change
```

במקום:

- experts
- exchange

המצב נהיה כל כך מבין, שהאתר שינה את כתובת ה-URL ל:

experts-exchange

כדי שיהיה ברור איך לקרוא את השם.

המסקנה היא ש-Casing חשוב מאוד, ולכן בעולם התכנות מקפידים עליו.

למה לא להשתמש בשמות כמו a, b, c

לדוגמה:

```
string a = string.Empty;  
string b = string.Empty;  
string c = string.Empty;
```

הקוד אמנם יעבוד, אבל אף אחד לא יבין מה כל משתנה מייצג.

אם נסדר אותם אחרת:

```
string b = string.Empty;  
string a = string.Empty;  
string c = string.Empty;
```

כבר אי אפשר לדעת:

- האם **a** הוא שם פרטי?
- האם **c** הוא שם משפחה?
- או להפך?

קוד נועד לא רק למחשב אלא גם לבני אדם שיקראו אותו.

לכן כדאי לבחור שמות ברורים שמתארים בדיוק מה המשתנה מייצג.

מצד שני, גם לא כדאי להגזים ולתת שמות ארוכים מדי, למשל:

```
string firstNameForTheUserWhoIsUsingOurApplication = string.Empty;
```

גם זה מקשה על הקריאה.

בנוסף, בדרך כלל לא משתמשים בקיצורים כמו:

```
string fName = string.Empty;
```

מפני שקיצורים ואקרונימים פחות ברורים לקוראים אחרים של הקוד.

```
string firstName = string.Empty;
```

הסטנדרט הוא:

```
string.Empty;
```

המשמעות של **string.Empty** היא שהמחרוזת ריקה ולא מאוחסן בה שום תוכן.

זוהי הדרך המקובלת לומר שרק התחלנו לעבוד עם המשתנה, ועדיין לא הכנסנו אליו שום ערך.

כלומר, זהו ערך התחלתי בלבד.

אפשר לכתוב:

```
string firstName = string.Empty;
```

או

אפשר לקצר את הקוד ולהכניס את הערך כבר בזמן יצירת המשתנה:

```
string firstName = "Tim";
```

זוהי מחרוזת שאין בה שום תוכן:

```
string firstName = "";
```

הכנסנו את הערך **"Tim"** למשתנה **firstName**.
כעת המשתנה **firstName** מחזיק את המידע **"Tim"**.

```
firstName = "Tim";
```

לאחר שהכנסנו את הערך למשתנה **firstName**, אפשר להדפיס אותו למסך, והפלט יהיה **"Tim"**.

```
Console.WriteLine(firstName);
```

כך מגדירים (מכריזים על) משתנה:

```
string firstName = string.Empty;
```

כך מקצים ערך למשתנה:

```
firstName = "Tim";
```

כך משתמשים במשתנה:

```
Console.WriteLine(firstName);
```

אפשר גם לכתוב:

```
firstName = "123";
```

אולי נראה ש-123 הוא מספר, אבל במקרה הזה הוא אינו מספר. המספר 123 הוא אכן מספר, אך כאשר הוא נמצא בין גרשיים כפולים (""), הוא הופך ל-**string**. כל דבר שנמצא בין "" נחשב למחרוזת.

אם נריץ את הקוד הבא:

```
string firstName = string.Empty;
```

```
firstName = "Tim";
```

```
firstName = "123";
```

```
Console.WriteLine(firstName);
```

כאשר כתבנו:

```
firstName = "Tim";  
firstName = "123";
```

נוצר מקום בזיכרון עבור "Tim" וגם עבור "123".
לאחר שהחלפנו את הערך, כבר לא משתמשים ב-string ה-"Tim",
ולכן המערכת צריכה לנקות אותה מהזיכרון.
לשם כך קיימת מערכת בשם **Garbage Collector**,
שעובדת ברקע באופן אוטומטי, מבלי שנצטרך להתעסק בה.

בפלט נראה רק את 123,
ולא נראה גם את Tim,
מפני שהערך החדש דרס את הערך הקודם.

ה- **Strings** הן אובייקטים יחסית יקרים מבחינת זיכרון.
כאשר שמים "Tim" בתוך **firstName**,
לא משנים את ה-string הקודם,
אלא נוצר ה-string החדש בזיכרון,
והמשתנה **firstName** מצביע אליו במקום אל הישן.

כאן אנחנו מחברים שלוש מחרוזות יחד.

```
" "
```

המחרוזת הזו נקראת **String Literal**.
כאן היא משמשת רק כדי להוסיף רווח בין השם הפרטי לשם המשפחה.

```
Console.WriteLine(firstName + " " + lastName);
```

אינטרפולציה של מחרוזות String Interpolation

אנחנו מוסיפים את הסימן \$ לפני ה-string.
המשמעות היא שניתן לשלב קוד #C בתוך ה-string עצמו.

```
Console.WriteLine($"Hello {firstName} {lastName}");
```

בפועל נוצר ה-string החדש:

```
"Hello {firstName} {lastName}"
```

כל מה שנמצא בתוך הסוגריים המסולסלים {}
נחשב לקוד #C,
ולא לטקסט רגיל.

בכל פעם שאנחנו משנים את ה-**string** או מבצעים עליו פעולה,
נוצר עותק חדש של ה-**string**.
לאורך זמן, פעולה זו עלולה להיות יקרה מבחינת ביצועים.

שורת הקוד שלנו היא:

```
filePath = "C:\\Temp\\Demo \\n";
```

מתחת ל-T\ ול-D\ נקבל קו אדום,
והודעת השגיאה תהיה:

```
unrecognized escape sequence.
```

רצף **Escape** לא מוכר.

לכאורה עשינו הכול נכון,
כי שמנו את הטקסט בין גרשיים,
אז למה מתקבלת שגיאה?

הבעיה היא שיש תווים מיוחדים ש-C# מפרש בצורה מיוחדת.
לכן קיים **Escape Character**.
לדוגמה:

```
\n
```

מייצג מעבר ל-**New Line**.

אם אנחנו רוצים שהתו \ עצמו יופיע כחלק מה-**string**,
עלינו לכתוב:

```
\\
```

כלומר שני תווי \.
הראשון אומר למחשב שהתו הבא הוא תו מיוחד,

והשני הוא התו שנרצה שיופיע בפועל ב-**string**.
אפשרות נוספת היא להשתמש בסימן **@** לפני ה-**string**:

```
@"
```

הסימן **@** נקרא **Verbatim String Literal**.
הוא אומר ל-**C#** להתייחס לכל התווים במחרוזת כתווים רגילים,
ולא לפרש אותם כרצפי **Escape**.
במצב כזה כבר לא ניתן להשתמש ב-**n** ובשאר רצפי ה-**Escape**.

אפשר גם לשלב את **\$** ואת **@** יחד.
כך ניתן להשתמש גם באינטרפולציה (**{firstName}**)
וגם לכתוב נתיב קובץ ללא צורך ב-****

```
string testString = @$"The file for {firstName} is at C:\SampleFile";
```

סימון הונגרי Hungarian Notation

```
string strFirstName = string.Empty;
```

כיום לא משתמשים בסגנון הזה,
ואף מעודדים שלא להשתמש בו בתעשייה,
למרות שהוא מציין את **type** המשתנה באמצעות התחילית **str**.

כאשר קוראים קוד כזה,
קשה יותר לקרוא את שם המשתנה.
במקום לראות מיד את **FirstName**,
המוח צריך קודם להפריד בין

```
str
```

לבין

```
FirstName
```


העצירה הקטנה הזו מאטה את הקריאה וההבנה של הקוד,
ולכן סגנון הכתיבה הזה כבר אינו מומלץ כיום.